# Library System

Software Design & Architecture — Group Activity

## 1. Candidate Conceptual Classes

Book, Member, Librarian, Library, Book Copy, Loan / Borrowing, Reservation, Fine, Author, Category, Membership, Payment.

## 2. Important Classes, Attributes & Methods

| Class | Attributes | Methods |
|---|---|---|
| Book | bookId, title, ISBN, author, category | searchBook(), updateDetails() |
| BookCopy | copyId, status, location | checkAvailability(), updateStatus() |
| Member | memberId, name, email, phone | borrowBook(), returnBook(), reserveBook() |
| Loan | loanId, issueDate, dueDate, returnDate | createLoan(), closeLoan(), calculateFine() |
| Reservation | reservationId, reservationDate, status | makeReservation(), cancelReservation() |
| Fine | fineId, amount, reason, status | calculateFine(), payFine() |
| Librarian | librarianId, name, email | issueBook(), receiveBook(), addBook() |

## 3. Abstraction Decisions

We ignore details that are not important for library operations, such as a member's complete home address, a book's physical weight or color, exact shelf dimensions, and unnecessary personal information about the librarian.

The system mainly focuses on borrowing, returning, reserving books, managing members, and calculating fines.

## 4. Invented Software Object

**Library Management Controller**

This software object may not exist as a physical real-world object, but it is useful for controlling library operations.

**Attributes:** systemId, currentDate
**Methods:** processBorrow(), processReturn(), checkAvailability(), calculateFine()

## 5. Main Relationships

Member → borrows → Loan
Loan → is for → BookCopy
Book → has → BookCopy
Member → makes → Reservation
Loan → may create → Fine
Librarian → manages → Library

## Conclusion

The Library System can be modeled using conceptual classes that represent important entities and responsibilities. Abstraction helps us keep only the details required by the system.